15 COPIAS

Índice del Apéndice del IDE FPC

IDE de Free Pascal (Integrated Development Environment) 2
El ciclo de desarrollo de un programa en el entorno Free Pascal IDE 2
Consideraciones para el trabajo en este ambiente de desarrollo 2
Permisos (para ejecución de IDE Free Pascal bajo Linux) 3
Archivos Ejecutables 4
Comenzar a trabajar en el IDE Free Pascal 4
El teclado y el manejo de textos en Free Pascal 10
Tratamiento de archivos en la ventana de edición 11
Secuencia de teclas y otras combinaciones 11
Sitios de Interés 13
Ejercicios adicionales 13
Códigos de error de compilación y ejecución 14

IDE de Free Pascal (Integrated Development Environment)

Un IDE (ambiente integrado de programación) es un programa que contiene, además del compilador (lenguaje de programación), utilidades y herramientas. Estos elementos están integrados, de modo que pueden llamarse fácilmente a cualquiera de los módulos, durante el proceso de programación.

Un entorno de programación típico contiene:

- Un editor, que proporciona el medio de introducir el texto y símbolos para crear código fuente.

- Un intérprete o un compilador, que convierte el código fuente en instrucciones que la computadora puede comprender y ejecutar.

- Un verificador de errores en tiempo de ejecución, que reconoce errores no detectados anteriormente en tiempo de compilación.

- Un depurador (debugger), que permite realizar seguimiento línea a línea de la ejecución del programa con el fin de analizar y corregir errores de estructura y/o lógicos.

El ciclo de desarrollo de un programa en el entorno Free Pascal IDE

Free Pascal es un compilador Pascal portable, libre y de código abierto, que representa una alternativa a los productos comerciales Turbo Pascal y Delphi. Su entorno en modo texto es muy similar al de Turbo Pascal de Borland, pero con más opciones.

Free Pascal (en sus últimas versiones: 2.2.0 y 2.2.2) es perfectamente compatible con Turbo Pascal 7.0 Free Pascal es un compilador para múltiples arquitecturas de CPU y múltiples sistemas operativos. El compilador implementa los dialectos de Pascal de Borland Turbo Pascal y Delphi, además de algunas construcciones de MacPascal. Está disponible para la mayoría de los sistemas operativos populares.

El compilador produce código máquina ejecutable independiente, es decir, que una vez que un programa se ha compilado en disco, puede transportarse el archivo ejecutable a otra máquina y ejecutarlo sin necesidad del IDE. El ciclo de desarrollo de un programa en este ambiente puede resumirse en los siguientes pasos:

1. Edición del programa fuente.
2. Almacenamiento del programa en un archivo. (Archivo de extensión .PAS)
3. Compilación del programa. Si existen errores de sintaxis, se debe volver a la edición del programa y corregirlos. En caso de no existir errores, tendremos un programa ejecutable.
4. Ejecución del programa (una vez que no haya errores de compilación)
5. Visualización de resultados. Si existen errores de lógica, entonces volver a la edición del programa fuente para modificaciones.

Consideraciones para el trabajo en este ambiente de desarrollo

- En la opción de menú **Compile—Target**, se debe seleccionar la plataforma adecuada. Por ejemplo, si está trabajando bajo Windows, se deberá seleccionar la opción **Win32 for i386**, en cambio si está trabajando bajo Linux deberá seleccionar **Linux for i386.** Esta opción del menú, nos

permite seleccionar otras arquitecturas como destino de la compilación, como por ejemplo (OS/2, FreeBsd/ELF for i386, etc).

- En la opción de menú **Options – Compiler—Syntax**, en la ventana correspondiente a **Compiler Mode** se debe seleccionar la opción **Turbo Pascal compatible.** Aquí en esta opción tenemos tambien la posibilidad de seleccionar distintas compatibilidades a otros codigos como ser Mcintosh Pascal, Object Pascal, etc.
- En la opción de menú **Code-Generation**, en la ventana **optimizations** permite entre otras seleccionar entre un código más rápido o uno más pequeño. (Generate faster code, Generate smaller code).
- En la opción de menú **Processor**, Donde se puede seleccionar los distintos Micro-procesadores. Ejemplo (Pentium tm, i386/i484, PentiumMMX, etc.
- En la opción **Verbose**, En la que se puede seleccionar distintos tipos información que retorne en la compilación por ejemplo los Warnings.
- También posee la opción **Browser** yla opción **Assembler**.

Como se puede observar el entorno es adaptable a varias plataformas y con distintas particularidades. Pero para el fin de nuestras necesidades basta con seleccionar las indicadas.

- Al compilar programas bajo Windows obtendremos archivos .EXE, en cambio, bajo Linux se genera un ejecutable sin extensión alguna.

Ejemplo:

Supongamos que tenemos almacenado un archivo EJEMPLO.PAS. Si se compila en *Windows* se genera "EJEMPLO.EXE" y en *Linux* se genera "EJEMPLO" (sin ninguna extensión). Éste último con permisos de ejecución por el dueño (usuario que compila). Si en *Linux* desde consola (shell) se ejecuta el comando "ls -l", equivalente al "dir" del DOS podremos ver:

Permisos	Dueño	Grupo	tamaño	Fecha	Hora	Nombre
-rwxr-xr-x	1 Juan	curso1	190007	2008-10-21	19:30	EJEMPLO
-rw-r--r--	1 Juan	curso1	1532	2008-10-21	19:30	EJEMPLO .O
-rw-r--r--	1 Juan	curso1	25	2008-10-21	19:30	EJEMPLO.PAS

Acerca de la información anterior podemos decir:
"EJEMPLO.PAS" es el Código o Programa Fuente. "EJEMPLO.O" es el Código Objeto y "EJEMPLO" es el Programa Ejecutable. "Juan" es el dueño de los tres archivos y pertenece al grupo"curso1".

Permisos (para ejecución de IDE Free Pascal bajo Linux)

Linux es un sistema multiusuario, esto quiere decir que en Linux cada usuario tiene sus elementos y otro usuario no puede acceder o modificarlas a menos que el primero le dé permisos. Para eso, cada archivo tiene un dueño, y una serie de permisos. El único autorizado a cambiar esos permisos es el dueño del archivo.

Los permisos básicos son los siguientes: Lectura (r), Escritura (w) y Ejecución (x). En el caso de los directorios, el permiso de ejecución significa que el usuario puede entrar o no al mismo.

A su vez, los usuarios pueden formar parte de un Grupo, de modo de compartir los permisos de un archivo o directorio. Entonces un grupo contiene uno o más usuarios, y un usuario puede formar parte de uno o más grupos.

De esta forma, los permisos de cada archivo se dividen en tres: los permisos para el dueño, para el grupo, y para el resto (o sea, los usuarios que no pertenecen a ninguna de las categorías anteriores).

En nuestro ejemplo: "EJEMPLO" tiene como permisos "rwxr-x-r-x" Esto significa que "Juan" (dueño) tiene todos los premisos "rwx", en cambio su grupo "curso1" y los otros sólo pueden leer y ejecutar, pero no modificar, porque tiene aplicado "r-x" en ambos casos.

Archivos Ejecutables

Como pudimos ver, una de las diferencias de los ejecutables entre los entornos DOS, Win32, NT, etc. y Linux es que en Linux no importa la extensión del archivo para saber si el sistema lo debe ejecutar. Si importan los permisos que el archivo contenga. El formato interno que genera para los ejecutables es del tipo **ELF**.

El formato **ELF** (Executable and Linkable Format) es un formato de archivo para ejecutables, código objeto, librerias compartidas y volcado de memoria. Fue desarrollado por el UNIX System Laboratory (USL), en principio fue desarrollado para plataformas de 32 bits, a pesar de que hoy en día se usa en gran variedad de sistemas. El formato ELF ejecutable es usado mayoritariamente en los sistemas tipo UNIX como GNU/LINUX, BSD, Solais, Irix. Existen otros formatos soportados en algunos de estos sistemas como COFF o a.out, pero ELF es sin duda el más usado.

Por otra parte, de la compilación para archivos ejecutables en DOS, Win32, etc. se pueden generar tanto .EXE como .COM. La principal diferencia es que el formato .EXE no restringe el tamaño del ejecutable, en cambio el .COM está restringido como máximo a ocupar un sólo segmento de memoria (un espacio de memoria de 64 Kb).

Cabe destacar que para ataques de seguridad es más tedioso revisar las cabeceras de los archivos que ver solamente su extensión.

Comenzar a trabajar en el IDE Free Pascal

TAREA 1

1.1 Editar del programa

Se lleva a cabo en una ventana de edición, en ella se puede escribir el programa fuente. Para comenzar a trabajar a la opción de menú **File – New.** Escribir las siguientes líneas de texto. Respetar la sintaxis propuesta

```
Program primero;
Begin
      Writeln ('Este es mi primer programa ');
      Writeln ('en el entorno de desarrollo para ');
      Writeln ('FREE PASCAL');
End.
```

1.2 Almacenar el programa editado en un archivo

Para almacenar el programa seleccionar la opción de menú **File -- Save As....** Elegir como nombre PRIMERO, no colocar extensión, ya que la misma se asignará de forma automática (.PAS). Click en OK

A continuación podrá observar que el nombre asignado por defecto NONAME.PAS ha sido cambiado por PRIMERO.PAS. Luego de cada cambio en el archivo fuente se recomienda su guardado accediendo a la opción **File -- Save** o **F2.** Cada nueva versión se almacenará reemplazando la anterior.

1.3 Compilar el programa:

Para compilar el programa seleccionar la opción de menú **Compile -- Compile (Alt F9).** Como ya se ha mencionado, en esta etapa se verifica que no existan errores sintácticos. Por cada error (la compilación termina de forma no exitosa) se debe efectuar la corrección correspondiente y recompilar hasta que la compilación sea exitosa.

1.4 Ejecutar el programa

Para obtener los resultados se ejecutará el programa mediante la opción de menú **Run – Run (Ctrl F9)**

1.5 Visualizar resultados

Una vez que se ejecutó el programa, ocurrirá que se vuelve a la ventana de edición sin haber podido ver los resultados. Para visualizar los resultados es necesario acceder a la "ventana del usuario" para lo cual deberá pulsar **Alt F5**

Verá en la pantalla:

Este es mi primer programa
en el entorno de desarrollo para
FREE PASCAL

1.6 Modificaciones

Agregar al final del programa (antes del **End**) la instrucción Readln. Almacene los cambios realizados **(F2).** Para ejecutarlo nuevamente es necesario volver a compilarlo, es decir se repiten los pasos 1.3 y 1.4.

Al realizar el paso 1.4, se observa que ahora para visualizar los resultados no es necesario pulsar **Alt F5** debido a la inclusión de la sentencia Readln. Una vez visualizados los resultados presione ENTER y vuelve a la ventana de Edición

TAREA 2

Escribir el siguiente programa en otra ventana de edición: Para esto seleccione **File—New**

```
Program rectangulo;
Uses  Crt;
Begin
  ClrScr;
  Writeln ('*******');
  Writeln ('*     *');
  Writeln ('*     *');
  Writeln ('*     *');
  Writeln ('*******');
  Readln;
End.
```

Sentencia Uses

La sentencia Uses permite identificar las Unidades (Units: son archivos compilados que pueden ser utilizados desde algún programa) que el programa podrá utilizar y significa que a partir de esa línea se podrán invocar todas las rutinas y/o tipos y/o constantes incluidas en dichas Unidades. El tema de unidades está explicado en el capítulo 7 del apunte de la materia.

Cada vez que se desee hacer uso de una Unit, debe tenerse en cuenta que ésta es un archivo independiente (no ejecutable), y como tal debe estar almacenado en alguna carpeta del disco rígido o unidad externa. Dicha carpeta o unidad debe estar explicitamente indicada en la ventana **Units** que aparece al seleccionar la opción de menú **Options – Directories...**

Crt es una Unit predefinida que contiene rutinas básicas de gestión de la pantalla en modo texto. Una de ellas es **ClrScr**. Este procedimiento limpia la pantalla del usuario, es decir elimina todo lo que haya sido impreso previamente en el monitor.

Si sólo se colocara la instrucción **ClrScr** sin poner la línea **Uses Crt**; la compilación daría error.

Otras rutinas en la unidad **Crt** son:

Nombre	Descripción
KeyPressed	Detecta la pulsación de una tecla
Sound	Suena el altavoz interno
GotoXY	Posiciona el cursor en las coordenadas de pantalla X,Y

Cuando se emplee más de una unit se tendrá que separarlas en comas. La línea siempre hay que finalizarla en punto y coma. Por ejemplo:

```
Uses Crt, Varios;
```

La Unidad **Printer** facilita la tarea del programador cuando ha de utilizar una impresora como dispositivo de salida. Permite enviar la salida del programa a la impresora utilizando *Write* o *Writeln* y **LST** como variable de texto para la salida. Por ejemplo:

```
Program Ejemplo;
Uses Printer;
Begin
  Writeln(LST,'Este texto aparece en la impresora');
  Writeln('Este texto aparece en pantalla');
End.
```

Importante
Al compilar una Unit llamada "miunidad.pas" en Free pascal para entornos Win32 se genera un "miunidad.tpu"; en cambio, en Linux genera un "miunidad.ppu".
Las unidades de TPU, son unidades PPU. Su funcionamiento es completamente idéntico; es decir se desarrollan igual y se las referencia de la misma manera que en TurboPascal.

Almacenar el programa editado en un archivo
Una vez editado se graba con el nombre SEGUNDO.PAS. En este momento se tienen abiertas dos ventanas al mismo tiempo.

Manejo de ventanas
Para alternar entre ventanas puede usarse la tecla **F6** o **Shift F6**. Si se conoce el número de la ventana a la que se quiere acceder (el número de cada ventana se encuentra en la esquina superior derecha), se puede acceder a ella mediante **Alt Nro.** O sea, para acceder a la ventana 3 se debe presionar **Alt 3**.

Para cerrar una ventana, se la debe activar (mediante alguna de las formas antes indicadas) y luego **Alt F3**

Compilación y ejecución

Se repiten los pasos 1.3 y 1.4 de la **Tarea 1**

A continuación se describen componentes del Editor de Free Pascal que serán de utilidad para el trabajo en la asignatura.

Componentes del editor de Free Pascal

La barra de menús es el sistema principal de acceso a todas las opciones de los menús. La barra de menú esta siempre visible, excepto cuando se está visualizando la salida de un programa.

Los distintos menús y sus opciones son las siguientes:

File Edit Search Run Compile Debug Tools Options Window Help

File	Edit	Search	Run
New	Undo	Find...	Run
New from template	Redo	Replace...	Step over
Open	Cut	Search again	Trace into
Reload	Copy	Go to line number...	Go to cursor
Save	Paste	Find procedure...	Until return
Save as ...	Clear	Objects	Run Directory
Save all	Select All	Modules	Parameters...
Print	Unselect	Globals	Program reset
Print Setup	Show Clipboard	Symbol	
Change Dir...	Copy to Windows		
Command Shell	Paste from Windows		
Exit			

Compile	Debug	Tools	Options
Compile	Output	Messages	Mode
Make	User Screen	Goto next	Compiler...
Build	Add watch	Goto previous	Memory sizes
Target	Watches	Grep	Linker...
Primary File...	Breakpoint	Calculator	Debugger...
Clear Primary File	Breakpoint list	ASCII Table	Directories
Compiler messages	Evaluate	.....	Browser
	Call stack		Tools
	.....		Enviroment

Las opciones de los menúes pueden ser de tres tipos distintos:

a.- Las órdenes que no requieren información adicional y que se ejecutan inmediatamente, por ejemplo: la orden **Run** del menú **Run**.

b.- La orden seguida por tres puntos suspensivos (...): Significa que cuando se ejecute esta orden se visualizará un cuadro de diálogo, que se utiliza para especificar la acción exacta que se desea ejecutar y la información adicional requerida.

c.- Las órdenes seguidas por una flecha (➢). En este caso, la orden conduce a otro menú.

Uso de Ventanas

Una ventana es una zona de la pantalla que se puede mover, redimensionar, solapar, abrir, cerrar y hacer zoom sobre ella.

Se pueden tener abiertas cualquier número de ventanas, pero sólo puede estar activada una sola ventana. La ventana activa es aquella en la que se está trabajando en ese momento. Las opciones del Menu **Window** son:

Orden	Descripción
Tile	Distribuye todas las ventanas abiertas en la pantalla, de forma que puedan visualizarse todas a la vez.
Cascade	Coloca en cascada las ventanas de edición abiertas.
Close All	Cierra todas las ventanas abiertas.
Size / Move	
Zoom	Se expande o reduce la ventana activa hasta el tama o del monitor.
Next	Pasa a la ventana siguiente.
Previous	Pasa a la ventana anterior.
Hide	Esconde la ventana. Para mostrarla accder a la lista de ventanas y alli seleccionar Show
Close	Cierra la ventana activa.
List...	Lista todas las ventanas abiertas.
Refresh display	Refresca o regenera la pantalla.

La línea de estado
La línea de estado aparece en la parte inferior de la ventana y sirve para visualizar las operaciones que pueden realizarse mientras se está escribiendo y editando programas en una ventana de edición.

Debugging (Depuración de Programas)

Es el proceso de identificar y corregir errores de programación. En inglés se le conoce como *debugging*, ya que se asemeja a la eliminación de *bichos* (*bugs*), manera en que se conoce informalmente a los errores de programación.
Si bien existen técnicas para la revisión sistemática del código fuente y se cuenta con medios computacionales para la detección de errores, sigue siendo en buena mèdida una actividad manual, que desafía la paciencia, la imaginación y la intuición del programador.

Algunas de las opciones de las que el IDE FPC dispone son:

F4	Run | Goto cursor	Ejecuta hasta la posición del cursor.
F7	Run | Trace Into	Ejecución del programa sentencia a sentencia
F8	Run | Step Over	Ejecución del programa, considerando las funciones o procedimientos como una sola sentencia
CTRL F2	Run | Program Reset	Corta el modo debugging

Ejemplo:

Al ejecutar el siguiente programa se detecta que entra en un ciclo infinito. La forma de encontrar el error, puede ser el debugging.

```
Program Ejemplo;
Var
   A, B : Integer;
   Op: Char;
Begin
   Writeln('Bienvenido');
   Op:='S';
   While Op <> 'N'do
   Begin
      Writeln('Ingrese dos valores enteros');
      Readln(A,B);
      If    (A > 0) and (B > 0) then
          Writeln( SQRT(A+B):6:2)
      Else
          Writeln(SQR(A+B):6:2)
   End
End.
```

Metodología:

Posicionarse en el renglón que contiene la sentencia `Op:='S';`. Hasta este punto el programa no tiene problemas. Presionar **F4**. Desde allí, realizar un seguimiento sentencia a sentencia con **F7** (en este caso, al no haber subprogramas, **F7** es equivalente a **F8**). De este modo podremos ver cómo se ejecuta el ciclo y deducir por qué no sale del While.
Si haciendo este seguimiento no es suficiente para reconocer el error, es posible incluir en una Ventana las variables utilizadas. Para esto seleccionar la variable (en nuestro caso *Op* y presionar **CTRL F7**) y observar su contenido, de este modo podremos ver que el ciclo se ejecuta de modo indefinido porque la variable *Op* nunca toma el valor N.

Si se desea cortar el Debugging antes de que termine la ejecución del programa, se debe presionar **CTRL F2**

El teclado y el manejo de textos en Free Pascal

Antes de comenzar a escribir programas es conveniente aprender el método de escritura de los mismos, es decir, la manipulación de textos.

Movimientos del cursor

Orden	Tecla Utilizada
Principio de línea	Home
Final de línea	End
Principio del archivo	CTRL Home
Final del archivo	CTRL End
Avanzar una página	Page Down
Retroceder una página	Page Up

Insertar y borrar texto

Descripción	Tecla Utilizada
Borrar caracter debajo del cursor	DEL
Borrar caracter a la izquierda del cursor	BACKSPACE
Modo Insertar | Sobreescritura	INS
Insertar línea	CTRL N
Borrar línea	CTRL Y

El modo de edición **Insertar** significa que los caracteres tipeados se insertan en el lugar donde se halla el cursor, desplazando el texto existente hacia la derecha. En el modo **Sobreescribir**, loque se escriba borrará lo que esté escrito a la derecha.

Ordenes de bloque de texto

Descripción	Orden	Orden Alternativa
Marcar Bloque	SHIFT y los cursores	
Borrar un bloque (va al portapapeles).	SHIFT DEL	Edit—Cut
Cortar un bloque (no va al portapapeles).	CTRL DEL	Edit—Clear
Pegar un bloque desde el portapapeles.	SHIFT INS	Edit—Paste
Copiar un bloque al portapapeles.	CTRL INS	Edit—Copy

Copiar un bloque:

1.- Marcar el bloque a repetir en otro lugar de la ventana de edición o bien en otra ventana.
2.- Seleccionar la opción Copy del menu Edit.
3.- Ubicar el cursor en el punto donde deseamos que aparezca el bloque elegido.
4.- Seleccionar del menu edit la opción Paste.

Mover un bloque

1.- Marcar el bloque a mover a otro lugar de la ventana de edición o bien a otra ventana.
2.- Seleccionar la opción Cut del menu Edit.
3.- Esta opción borro el bloque de texto, y ádemas lo coloca en el portapapeles.
4.- Ubicar el cursor en el punto donde deseamos que aparezca el bloque elegido.
5.- Seleccionar del menu Edit la opción Paste.

Borrar un bloque

1.- Marcar el bloque a eliminar.
2.- Seleccionar la opción Clear del menu Edit.

Se puede utilizar también la opción Cut. La diferencia entre una y otra es que Clear borra el bloque de texto y no lo mantiene en el portapapeles, es decir lo elimina sin posibilidad posterior de recuperación.

Menú Search

El menú Search contiene ordenes para encontrar textos específicos o ciertos tipos de texto (tales como errores en tiempo de ejecución).

Orden	Descripción
Find...	Encuentra el texto especificado en el cuadro de dialogo.
Replace...	Busca el texto especificado y lo sustituye por otro.
Search again	Repite la última orden Find... o Replace...
Go to line number...	Se desplaza al número de línea especificado en el cuadro de dialogo.
Show last compiler error...	Muestra el último error de compilación.
Find error...	Encuentra la posición del código fuente correspondiente a su error en tiempo de ejecución.
Find procedure...	Encuentra el principio del procedimiento especificado en el cuadro de dialogo.

Tratamiento de archivos en la ventana de edición

El menú File contiene órdenes para crear, abrir, cerrar e imprimir archivos. Para desplegar este menú, pulsar ALT F. Algunas de las órdenes más importantes son:

Orden	Descripción
New	Crea un archivo nuevo en una ventana de edición. El archivo tiene nombre NONAMExx.PAS (xx representa una par de digitos). Para que lo editado quede verdaderamente guardado es necesario usar la orden **Save** o **Save As...**
Open...	Abre un archivo existente y lo carga en una ventana de edición.
Save	Guarda el archivo actual con todos los cambios realizados y con el nombre actual. Si no fue almacenado previamente, esta opción es equivalente a **Save**.
Save As...	Permite guardar el archivo actual con otro nombre. Cuando se utiliza este comando los cambios realizados quedan almacenados en el nuevo archivo y el original mantendrá el mismo contenido que poseía antes de ser abierto.
Save All	Guarda todos los cambios realizados en todos los archivos abiertos.
Change dir...	Permite cambiar el subdirectorio actual
Print	Imprime el contenido de la ventana de edición activa.
Printer setup...	Permite configurar la impresora.
Command Shell	Realiza una salida temporal hacia la linea de comandos. Se puede volver al IDE tipeando exit en la línea de comandos.
Exit	Salida definitiva del entorno Free Pascal.

Recomendamos que con frecuencia pulse la tecla F2 durante la edición de los programas. Esta medida de precaución le evitará pérdidas accidentales de sus programas, por cortes imprevistos.
La tecla F2 realiza un guardado automático del contenido de la ventana de edición.

Secuencia de teclas y otras combinaciones

Teclas de función

Secuencia de teclas	*Menú equivalente*	*Función*
F1	Help	Visualiza una pantalla de ayuda.
F2	File | Save	Guarda archivo activo del editor.
F3	File | Open	Abre un archivo.
F5	Window | Zoom	Hace zoom de la ventana activa.
F6	Windows | Next	Pasa a otra ventana abierta.
F9	Compile | Make	

Otras Combinaciones

Secuencia de teclas	*Menú equivalente*	*Función*
CTRL F9	Run | Run	Ejecuta el programa que se encuentra en la ventana activa
ALT F5	Window | User screen	Visualización de la pantalla del usuario
ALT X	File | Exit	Sale de Free Pascal
ALT 0	Windows | List	Visualiza lista de ventanas abiertas
ALT N	Ninguno	Visualiza ventana número N

Sitios de Interés

-Se puede descargar Free Pascal de la siguiente dirección : www.freeepascal.org
-Sitios con ejemplos ilustrados de creación, edición, compilación y ejecución:
http://www.fing.edu.uy/inco/cursos/prog1/pm/field.php/Laboratorio/Compilador2008?from=Laboratorio.Compilador (ir a la sección 3.2)
http://es.youtube.com/watch?v=LihWEUNJ25k (video)
http://es.youtube.com/watch?v=R0HXdnj0weg (video)

Ejercicios adicionales

1.- a) Editar el siguiente programa:

```
Program Calcular_Area_del_triangulo;
var
 base,altura:real;
begin
   base:=2.4;
   altura:=5.3;
   writeln(0.5*base*altura);
end.
```

b) Almacenar en una carpeta adecuada el archivo fuente
c) Compilar el archivo fuente
d) Ejecutar.
e) Agregarle las siguientes líneas luego de `writeln`:

```
write('Ingrese base  ');
readln(base);
write('Ingrese altura  ');
readln(altura);
writeln('El area el triangulo con base ', base,' y altura' ,
                        altura, 'es = ', 0.5*base*altura);
```

f) repetir los incisos b) a d) con las modificaciones introducidas.

2.- a) Editar el siguiente programa:

```
Program Calcular;
var
 r, n : integer;
begin
  Write('R=')
  Readln( r);
  Write('N=')
  Readln( n);
  Write('Resultado:')
  Writeln( (r*/n-1) div (r-1));
end.
```

b) Almacenar en una carpeta adecuada el archivo fuente.
c) Compilar el archivo el archivo fuente.
d) Si tiene errores de sintaxis, modificarlo a fin de eliminar los errores (aquí no importa el resultado), repitiendo los incisos b y c)
e) Ejecutar el archivo

CODIGOS DE ERRORES

Significado de los errores más comunes de Compilación en Free Pascal

Identifire not found "aaa".
La variable "aaa" no está definida.

Sintax error ";" expected.
Falta un ";" separador de sentencias.

";" expected but "else" found.
Antes del "else" no va ";".

Durplicate identifier "aaa".
Se ha definido dos variables con el mismo nombre "aaa".

String exceeds line.
Omitió la comilla de fin de comentario.

Incompatible types : go "aaa" expected "bbb"
Tipos incompatibles, intenta asignar un valor de tipo "aaa" a una variable de tipo "bbb".

Wrong number of parameters specified for call to "aaa".
Se invoca un procedimiento "aaa" con un número incorrecto de parámetros.

Call by var for arg nº X has too match exactly ...
Se invoca un procedimiento con parámetros de distinto tipo.

Expected Incompatible types : got "untyped" "aaa".
Se produce al invocar un procedimiento como función. Se intenta asignar el procedimiento a una variable de tipo "aaa". El procedimiento no devuelve ningún tipo.

":" expected but " ; " found.
Se omitieron los ":". Ocurre cuando al definir una función se omite el tipo de ésta.

". " Expected but "end of file" found.
Se omitió el "." final del programa.

";" expected but " . " found.
Se encontró "." en vez de ";". Ocurre cuando no todos los "begin" tienen su "end" correspondiente.

"Then" expected but "aaa" found.
Se omitió la palabra reservada "then" .

Códigos de errores de ejecución

1. Número de función inválida. Se intentó una llamada no válida al sistema operativo
2. Archivo no encontrado. Se trató de borrar, renombrar o abrir un archvio que no existe.
3. Camino no encontrado.
4. Demasiados archivos abiertos. Su proceso alcanzó el máximo de archivos abiertos permitidos.
5. Denegado el acceso al archivo. Puede ser causado por:
   - Tratar de abrir para escritura un archivo que es sólo de lectura
   - El archivo está actualmente bloqueado o está siendo usado en otro proceso.
   - Tratar de crear un archivo o directorio con el mismo nombre de uno que ya existe.
   - Tratar de leer desde un archivo que fue abierto en el modo de sólo lectura

- Tratar de escribir en un archivo que fue abierto como de sólo escritura.
- Tratar de eliminar un archivo o directorio cuando no es posible.
- No existe permiso para acceder al archivo o directorio.

6. Manejo inválido de archivo. Si esto ocurre, la variable archivo que está usando será eliminada; indica que la memoria está corrupta.

12. Acceso no válido al archivo. Es reportado cuando un reset o rewrite es llamado un valor no válido de FileMode

15. Número de drive no válido

16. No se puede eliminar el directorio actual. Es rportado cuando se trata de eliminar el directorio que está activo.

17. No se puede renombrar el archivo.

100. Error en la lectura desde el disco. Sucede cuando se trata de leer pasado el final del archivo

101. Error en la escritura en el disco. Es reportado cuando el disco está lleno y se está intentando escribir en él.

102. El archivo no fue asignado. Es reportado al invocar Reset, Rewrite, Append, Rename y Erase, con un archivo no asignado.

103. Archivo no abierto. Es reportado por las siguientes funciones: Close, Read, Write, Seek, EOf, FilePos, FileSize, Flush, BlockRead, and BlockWrite

104. Archivo no abierto para entrada, no fue abierto con Reset. Es reportado por: Read, BlockRead, Eof, Eoln, SeekEof or SeekEoln

105. Archivo no abierto para salida, no fue abierto con Rewite.

106. Formato numérico no válido. Es reportado cuando se lee, desde un archivo de texto, un valor no numérico y se espera un valor numérico.

150. El disco está protegido para escritura (Error crítico)

151. Tamaño erroneo en la estructura de petición al dispositivo

152. El dispositivo no está listo. (Error crítico)

154. Error de CRC en los datos (Error crítico)

156. Error de búsqueda en el disco (Error crítico)

157. Tipo de medio desconocido (Error crítico)

159. Sector no encontrado (Error crítico)

159. Impresora sin papel (Error crítico)

160. Fallo de escritura en dispositivo (Error crítico)

161. Fallo de lectura en dispositivo (Error crítico)

162. Fallo en el hardware (Error crítico)

200. División por cero

201. Error en la comprobación de rango. Si se compiló el programa con controlador de rango, este error se produce en los siguientes casos:

1. Se accedió a un arreglo con un índice que está fuera del rango declarado para el arreglo
2. Se intentó asignar un valor a una variable fuera de su rango (por ejemplo en un tipo enumerado)

202. Error de desbordamiento de pila. Este error se produce sólo si el controlador de pila está habilitado.

204. Operación de puntero no válida.

205. Desbordamiento de punto flotante. Está tratando de usar o producir números reales demasiado grandes.

206. Subdesbordamiento de punto flotante. Está tratando de usar o producir números reales demasiado pequeños.

207. Operación de punto flotante no válida. Puede ocurrir al intentar calcular la raíz cuadrada o el logaritmo de un número negativo.